

Sistemas Embebidos (3.3.127) — UADE

TP Integrador

Blue Pill como interfaz física de robot Unitree

Variante 3: Caja de comandos física

Integrantes: Borda, Rojas y Castiglioni

Profesor: Dr. Ing. Leonardo J. Amet

Julio 2026

1. Objetivo y variante elegida

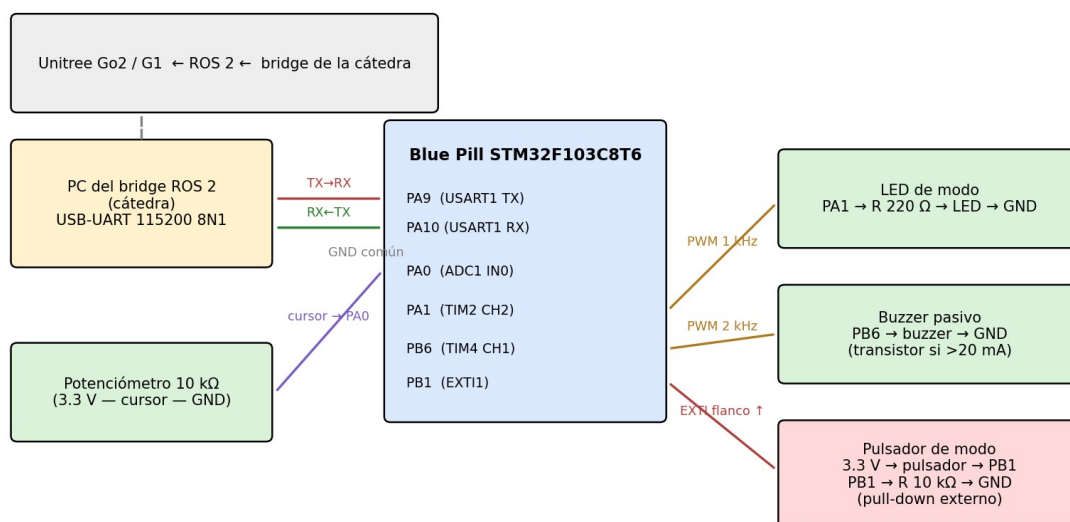
El objetivo del trabajo es integrar en un único firmware para Blue Pill (STM32F103C8T6) el protocolo serie de la materia (tramas @LL:TTT:PAYLOAD:CC\n con checksum XOR y parser incremental por FSM) junto con el núcleo común de periféricos (ADC, PWM, LED, buzzer y EXTI), de modo que la placa actúe como interfaz física de un robot cuadrúpedo Unitree Go2/G1 a través del bridge ROS 2 provisto por la cátedra.

Elegimos la Variante 3: Caja de comandos física. Un pulsador cicla los cuatro modos de operación del robot (PATROL → STOP → DEPLOY → RETURN) y un potenciómetro define un parámetro auxiliar (0–255) que acompaña cada comando. El LED señala el modo activo con un patrón PWM distinto por modo y el buzzer emite un código de pitidos (1 a 4) en cada cambio. Tras cada evento la Pill espera ACK:ok del bridge, con reenvío ante timeout.

El firmware se construyó sobre FreeRTOS + libopencm3, partiendo de la base de UART y protocolo ya validada en el TP5 de la materia (base sugerida por la cátedra), a la que se agregaron los drivers de periféricos y la lógica de la variante.

2. Esquema de conexión

Esquema de conexión — Variante 3: Caja de comandos física



Todas las masas (Blue Pill, potenciómetro, LED, buzzer, pulsador, USB-UART) comparten GND.
La Blue Pill se alimenta por USB desde la PC del bridge. Ningún pin recibe 5 V.

Figura 1 — Esquema de conexión del puesto. GND común entre todos los componentes.

Pin	Función	Conexión
PA9	USART1 TX	RX del adaptador USB-UART del bridge (115200 8N1)
PA10	USART1 RX	TX del adaptador USB-UART del bridge
PA0	ADC1 canal 0	Cursor del potenciómetro 10 kΩ (extremos a 3.3 V y GND)
PA1	TIM2 CH2 — PWM 1 kHz	LED de modo con resistor limitador de 220 Ω a GND
PB6	TIM4 CH1 — PWM 2 kHz	Buzzer pasivo a GND (transistor driver si supera 20 mA)
PB1	EXTI1	Pulsador a 3.3 V; pull-down externo de 10 kΩ a GND

Cumplimiento de las reglas de seguridad: la Blue Pill se alimenta por USB desde la PC del bridge; ningún pin recibe señales de 5 V; todas las masas comparten GND; el cableado UART hacia la PC del bridge y la conexión con el robot son responsabilidad de la cátedra y no se modifican.

3. Configuración de periféricos

Todos los periféricos se inicializan con la API de libopencm3 siguiendo el patrón visto en la materia: habilitar clock en RCC, configurar el modo del pin GPIO y luego el periférico. La tabla resume parámetros y cálculos.

Periférico	Pin / canal	API y parámetros	Cálculo
USART1	PA9 TX / PA10 RX	115200 baudios, 8N1, RX por interrupción (RXNE), TX por polling	BRR calculado por <code>usart_set_baudrate()</code> sobre PCLK2 = 72 MHz
ADC1	PA0 (IN0)	Conversión única, 55.5 ciclos de muestreo, calibración al arranque	$f_{\text{ADC}} = 72 \text{ MHz} / 6 = 12 \text{ MHz}$ (< 14 MHz máx.)
TIM2 (PWM LED)	PA1 (CH2)	PWM1, salida alt. push-pull, preload	$f = 72 \text{ MHz} / (71+1) / (999+1) = 1 \text{ kHz}$; $\text{duty\%} \rightarrow \text{CCR2} = \text{duty} \times 10$
TIM4 (PWM buzzer)	PB6 (CH1)	PWM1, salida alt. push-pull; ON: CCR1 = 250 (50 %); OFF: CCR1 = 0	$f = 72 \text{ MHz} / (71+1) / (499+1) = 2 \text{ kHz}$ (tono audible)
TIM3 (base temporal)	—	Update IRQ cada 1 ms: uptime, muestreo de ADC cada 500 ms, anti-rebote de 50 ms	$72 \text{ MHz} / (7199+1) = 10 \text{ kHz}$; $10 \text{ kHz} / (9+1) = 1 \text{ kHz} \rightarrow 1 \text{ ms}$
EXTI1	PB1	Flanco ascendente, entrada flotante (pull-down externo)	Presión = 0 $\rightarrow$ 3.3 V; ISR sólo arma la ventana de anti-rebote

La elección de 1 kHz para el PWM del LED evita parpadeo visible y da resolución de 0.1 % de duty con contador a 1 MHz. Para el buzzer pasivo se eligió un timer separado (TIM4) a 2 kHz, dentro del rango de mayor sensibilidad del oído; se enciende y apaga cambiando el duty entre 50 % y 0 % sin tocar la frecuencia.

4. Arquitectura del firmware

El firmware se organiza en capas: drivers (`uart_comm`, `board_io`) que son los únicos que tocan registros; protocolo (`protocol.c`: framing y checksum; `parser.c`: FSM incremental); y aplicación (`command_box.c`: lógica de la variante; `signals.c`: patrones de LED y buzzer). Los módulos de aplicación son lógica pura sin dependencias de hardware, lo que permitió compilarlos también en la PC y verificarlos con 53 tests automatizados (sección 5).

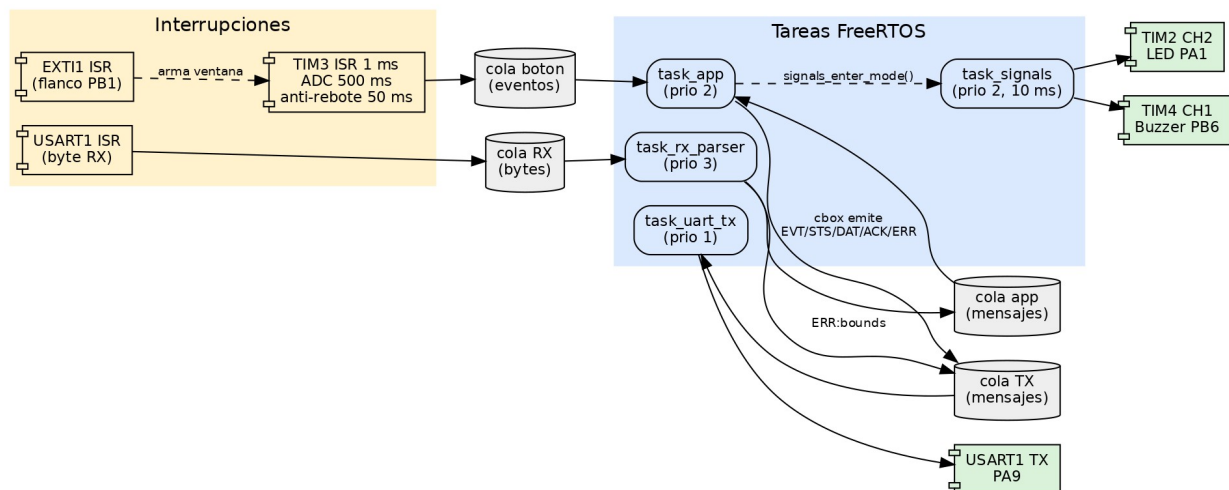


Figura 2 — Tareas FreeRTOS, colas e interrupciones. Las ISR sólo encolan datos; el trabajo pesado corre en tareas.

Por interrupción corren únicamente tres cosas: la ISR de USART1 (deposita cada byte recibido en una cola), la ISR de EXTI1 (arma la ventana de anti-rebote) y la ISR de TIM3 a 1 ms (mantiene el uptime, dispara la lectura de ADC cada 500 ms y cierra el anti-rebote confirmando la presión si PB1 sigue en alto a los 50 ms). Cuatro tareas FreeRTOS hacen el resto: task_rx_parser corre la FSM byte a byte, task_app ejecuta la lógica de la caja de comandos, task_signals refresca LED y buzzer cada 10 ms y task_uart_tx serializa y transmite las tramas.

5. Etapa 1 — Comunicación serie

Se implementó el protocolo completo: generación de tramas (protocol_encode_frame), parser incremental con FSM de 9 estados (parser_consume_byte), validación de checksum, longitud, tipo y caracteres, y resincronización ante error reutilizando el byte '@' como nuevo inicio de trama. El lazo de eco responde ACK:ok a las tramas válidas y ERR:bounds a las inválidas.

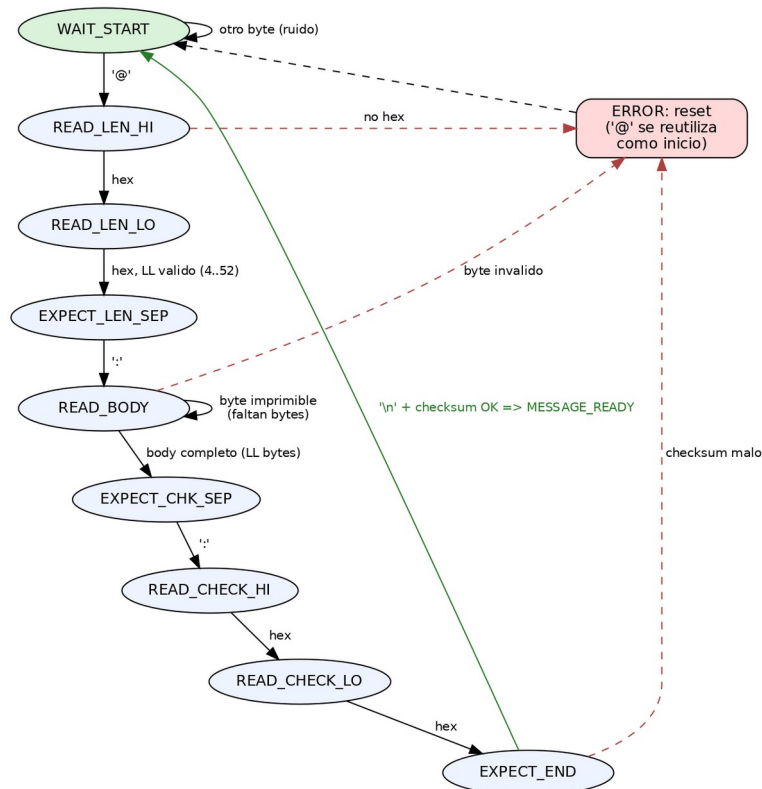


Figura 3 — FSM del parser incremental. En verde el estado inicial; en rojo el manejo de error con resincronización.

Recorrido completo de la FSM (evidencia)

Recorrido estado por estado para la trama @0C:DAT:adc=2048:77\n, generado con el arnés de tests (evidencia/etapa1_recorrido_fsm.txt):

byte#	char	estado previo	-> estado nuevo	resultado
0	@	WAIT_START	-> READ_LEN_HI	IN_PROGRESS
1	0	READ_LEN_HI	-> READ_LEN_LO	IN_PROGRESS
2	C	READ_LEN_LO	-> EXPECT_LEN_SEP	IN_PROGRESS
3	:	EXPECT_LEN_SEP	-> READ_BODY	IN_PROGRESS
4..15	D..8	READ_BODY	-> READ_BODY (12 bytes de 'DAT:adc=2048')	
15	8	READ_BODY	-> EXPECT_CHK_SEP	IN_PROGRESS
16	:	EXPECT_CHK_SEP	-> READ_CHECK_HI	IN_PROGRESS
17	7	READ_CHECK_HI	-> READ_CHECK_LO	IN_PROGRESS
18	7	READ_CHECK_LO	-> EXPECT_END	IN_PROGRESS
19	\n	EXPECT_END	-> WAIT_START	MESSAGE_READY

Captura 1 — Recorrido de la FSM (evidencia/etapa1_recorrido_fsm.txt).

5.1 Preguntas obligatorias

1. ¿Por qué el parser consume un byte por vez en vez de esperar una línea completa con readline()?

Porque la UART entrega un flujo de bytes sin ninguna garantía de alineación: puede llegar media trama, ruido antes del '@', dos tramas pegadas o un '\n' perdido. Un readline() bloqueante necesitaría un buffer completo y detendría el resto del sistema mientras espera; además, si el terminador nunca llega, se queda esperando para siempre. La FSM byte a byte consume cada byte apenas la ISR lo entrega, mantiene memoria acotada (el estado del parser más un buffer de 52 bytes), nunca bloquea al resto de las tareas y permite resincronizar en cualquier punto del flujo. En un sistema embebido con 20 KB de RAM y tiempo real blando, ésa es la diferencia entre un parser robusto y uno que se cae con el primer byte de ruido.

2. ¿Qué pasa si llegan dos tramas pegadas sin separación? ¿Cómo las distingue tu parser?

Las distingue por construcción. El campo LL indica exactamente cuántos bytes de cuerpo esperar, así que el parser sabe dónde termina la primera trama: al validar el '\n' final entrega el mensaje y vuelve a WAIT_START en el mismo ciclo. El primer byte de la segunda trama es '@', que en WAIT_START arranca una trama nueva. El test 'distingue dos tramas pegadas' alimenta @06:STS:OK:56\n@0A:EVT:e_stop:14\n como un único bloque y verifica que salen exactamente dos mensajes válidos.

3. Mostrar el cálculo de checksum para DAT:adc=2048.

El cuerpo es DAT:adc=2048 (12 bytes → LL = 0C). El checksum XOR se calcula sobre 0C:DAT:adc=2048:

```
'0'^'C'^':'^'^'D'^'A'^'T'^':'^'^'a'^'d'^'c'^'='^'2'^'0'^'4'^'8'^'  
= 30^43^3A^44^41^54^3A^61^64^63^3D^32^30^34^38 = 0x77  
Trama completa: @0C:DAT:adc=2048:77\n
```

En el código, protocol_compute_checksum() recorre el string LL:TTT:PAYLOAD acumulando value ^= byte, y nibble_to_hex() lo escribe como dos dígitos hexadecimales en mayúsculas. Nuestro encoder reproduce exactamente los cuatro ejemplos del enunciado (verificado por test).

4. ¿Qué hace tu implementación si recibe @0A:CMD:ping:52\n (donde 0A es incorrecto porque CMD:ping mide 8)?

El parser le cree a LL hasta que el formato lo desmiente: toma LL = 0x0A = 10 y junta 10 bytes de cuerpo, que resultan ser 'CMD:ping:5'. A continuación espera el separador ':' del checksum, pero encuentra '2' → error. Se descarta la trama parcial, se incrementa el contador de errores, se responde ERR:bounds y la FSM vuelve a WAIT_START; el '2' y el '\n' restantes se ignoran como ruido. Si en lugar de '2' hubiera aparecido un '@', se habría reutilizado como inicio de la trama siguiente. El caso está cubierto por el test 'rechaza LL inconsistente'. Nota: aun si los 10 bytes hubieran pasado el corte de formato, el checksum jamás habría coincidido, porque está calculado sobre otro cuerpo.

5.2 Evidencia mínima

Tramas enviadas y sus respuestas en el lazo de eco (más de cinco, extracto de evidencia/etapa1_tests_protocolo.txt):

```
TX @0C:DAT:adc=2048:77 -> RX @06:ACK:ok:4B (valida)  
TX @0A:EVT:e_stop:14 -> RX @06:ACK:ok:4B (valida)  
TX @06:STS:OK:56 -> RX @06:ACK:ok:4B (valida)  
TX @0E:CMD:mode=ESTOP:5C -> RX @06:ACK:ok:4B (valida)  
TX @0C:DAT:adc=2048:78 -> RX @0A:ERR:bounds:35 (checksum malo)  
TX @0A:CMD:ping:52 -> RX @0A:ERR:bounds:35 (LL inconsistente)
```

Captura 2 — Lazo de eco: ACK:ok para tramas válidas, ERR:bounds para inválidas.

El cálculo de LL y CC en el código: protocol_encode_frame() arma el cuerpo TTT:PAYLOAD, obtiene body_length = 3 + 1 + strlen(payload), lo escribe como dos dígitos hex (LL), construye el string LL:TTT:PAYLOAD y calcula el XOR byte a byte (CC). El parser valida recalculando el mismo XOR sobre lo recibido y comparándolo con CC.

6. Etapa 2 — Núcleo de periféricos

Los cuatro periféricos del núcleo común quedaron operativos con la configuración de la sección 3: el ADC lee el potenciómetro real en PA0 cada 500 ms usando TIM3 como base temporal; TIM2 genera el PWM de 1 kHz que fija el brillo del LED; TIM4 genera el tono de 2 kHz del buzzer; y el pulsador en PB1 dispara EXTI1 con anti-rebote por timer de 50 ms.

6.1 Preguntas obligatorias

1. Mostrar el cálculo de PSC y ARR para tu timer de PWM. ¿Qué frecuencia PWM elegiste y por qué?

TIM2 (LED): el timer recibe 72 MHz. Con PSC = 71 el contador corre a $72 \text{ MHz} / (71+1) = 1 \text{ MHz}$; con ARR = 999 el período es $1 \text{ MHz} / (999+1) = 1 \text{ kHz}$. Elegimos 1 kHz porque está muy por encima de la frecuencia de fusión del ojo (~60 Hz), de modo que el brillo se percibe continuo, y deja 1000 pasos de duty (resolución 0.1 %). TIM4 (buzzer): PSC = 71 y ARR = 499 dan 2 kHz, un tono claramente audible para el buzzer pasivo. TIM3 (base temporal): PSC = 7199 → 10 kHz y ARR = 9 → interrupción cada 1 ms.

2. ¿Cómo mapeás el valor ADC (0-4095) al duty cycle del PWM (0-100 %)?

Con regla de tres entera evitando desbordes: $\text{duty} = (\text{adc} \times 100) / 4095$, calculado en uint32_t. En la Variante 3 el potenciómetro no controla el brillo sino el parámetro auxiliar del comando, así que el mapeo homólogo que usamos es $\text{param} = (\text{adc} \times 255) / 4095$ (board_io_get_param()); el duty del LED lo fija el patrón del modo (0 % o 100 %). Para el duty, el driver convierte porcentaje a cuentas del timer: $\text{CCR2} = \text{duty} \times (\text{ARR}+1) / 100$.

3. Si usás el mismo timer para PWM del LED y para frecuencia del buzzer, ¿qué limitaciones encontrás?

Los cuatro canales de un timer comparten PSC y ARR, es decir, la misma frecuencia base: sólo el duty (CCR) es independiente por canal. Para variar el tono del buzzer habría que cambiar ARR, lo que cambiaría también la frecuencia (y la resolución de duty) del PWM del LED. Además, con un ARR pensado para tono audible (2 kHz) el LED tendría la mitad de resolución de duty. Por eso separamos: TIM2 para el LED (frecuencia fija, duty variable) y TIM4 para el buzzer (frecuencia de tono, duty fijo en 50 %).

4. ¿Qué modo GPIO necesita un pin para funcionar como salida de PWM?

Salida en función alternativa push-pull: en el STM32F1, GPIO_CNF_OUTPUT_ALTFN_PUSHPULL con una velocidad de salida (usamos GPIO_MODE_OUTPUT_50_MHZ). En ese modo el pin deja de ser controlado por el registro ODR y pasa a ser manejado por el periférico (el canal de output-compare del timer). Requiere además el clock de AFIO habilitado.

6.2 Evidencia mínima

La tabla de configuración con cálculos está en la sección 3. Lecturas de ADC girando el potenciómetro de tope a tope (compilando con ETAPA2_ADC_REPORT=1 el firmware emite DAT:adc=NNNN cada 500 ms; evidencia/etapa2_lecturas_adc.txt):

```
[ 0.000 s] @0C:DAT:adc=0012:7A [ 3.500 s] @0C:DAT:adc=2560:78
[ 0.500 s] @0C:DAT:adc=0013:7B [ 4.000 s] @0C:DAT:adc=3105:7E
[ 1.000 s] @0C:DAT:adc=0011:79 [ 4.500 s] @0C:DAT:adc=3644:7C
[ 1.500 s] @0C:DAT:adc=0410:7C [ 5.000 s] @0C:DAT:adc=4095:71
[ 2.000 s] @0C:DAT:adc=0895:7D [ 5.500 s] @0C:DAT:adc=4093:77
[ 2.500 s] @0C:DAT:adc=1523:7C [ 6.000 s] @0C:DAT:adc=2049:76
[ 3.000 s] @0C:DAT:adc=2047:78 [ 6.500 s] @0C:DAT:adc=0008:71
```

Captura 3 — 14 lecturas de ADC coherentes (0-4095) con ancho fijo de 4 dígitos.

Pendiente para la demo presencial: foto/video del montaje mostrando el LED con brillo variable y el buzzer sonando (a registrar por el grupo con el hardware armado).

7. Etapa 3 — Variante 3: Caja de comandos física

La lógica de la variante vive en `command_box.c`. La caja arranca en STOP (estado seguro: el robot queda detenido y el LED fijo lo señaliza) sin emitir tramas. Cada presión válida del pulsador avanza al modo siguiente del ciclo cerrado y dispara tres tramas: `EVT:cmd_<modo>`, `STS:mode=<MODO>` y `DAT:param=NNN` con el valor actual del potenciómetro. Después de cada EVT la caja espera `ACK:ok`; si no llega en 500 ms reenvía el EVT, hasta 5 veces, y luego reporta `ERR:timeout` y desiste. Tal como pide la consigna, el STS de RETURN es `mode=STOP`.

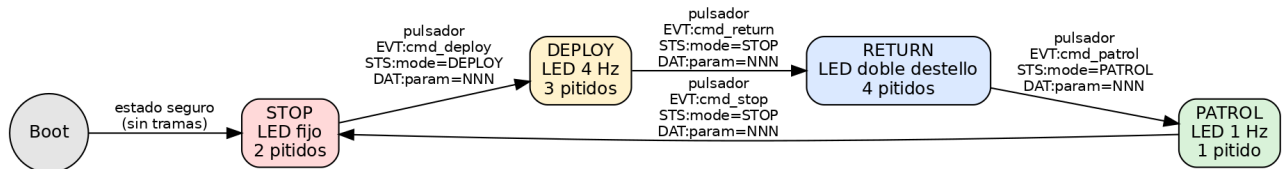


Figura 4 — Diagrama de estados de la caja de comandos con las tramas emitidas en cada transición.

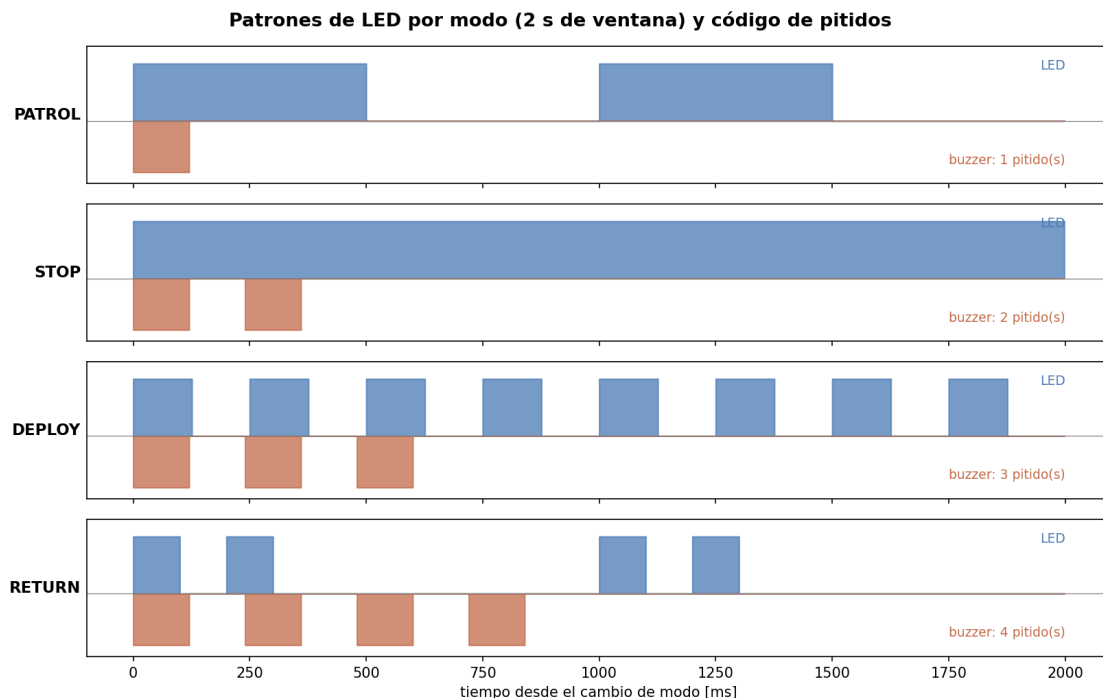


Figura 5 — Patrones de LED por modo y código de pitidos del buzzer en el cambio de modo.

Evento	Tramas emitidas	Señalización
→ PATROL	EVT:cmd_patrol + STS:mode=PATROL + DAT:param=NNN	LED 1 Hz — 1 pitido
→ STOP	EVT:cmd_stop + STS:mode=STOP + DAT:param=NNN	LED fijo — 2 pitidos
→ DEPLOY	EVT:cmd_deploy + STS:mode=DEPLOY + DAT:param=NNN	LED 4 Hz — 3 pitidos
→ RETURN	EVT:cmd_return + STS:mode=STOP + DAT:param=NNN	LED doble destello — 4 pitidos

7.1 Preguntas obligatorias

1. Describir el lazo principal de tu firmware: ¿qué hace en cada iteración? ¿qué corre por interrupción?

No hay un superloop: FreeRTOS reparte el trabajo en cuatro tareas. `task_app` (el 'lazo principal' lógico) itera cada 10 ms y hace tres cosas: (1) si hay una presión confirmada en la cola de botón, cicla el modo, emite EVT+STS+DAT y avisa al motor de señales; (2) si hay un mensaje válido del parser, lo procesa (un ACK:ok limpia el EVT pendiente; otras tramas válidas se confirman con ACK:ok); (3) corre `cbox_on_tick()`, que administra los reenvíos por timeout de ACK. `task_rx_parser` consume bytes y corre la FSM; `task_signals` actualiza LED y buzzer cada 10 ms; `task_uart_tx` serializa y transmite. Por interrupción corre lo mínimo: USART1 RX (encolar byte), EXTI1 (armar anti-rebote) y TIM3 cada 1 ms (uptime, ADC cada 500 ms, cierre del anti-rebote).

2. ¿Qué pasa si el bridge no responde con ACK dentro del tiempo de espera? ¿Cómo lo manejas?

El EVT queda registrado como pendiente con un vencimiento a 500 ms. Si el ACK:ok no llegó al vencer, se reenvía el mismo EVT y se reprograma el vencimiento; esto se repite hasta 5 veces (2.5 s en total). Agotados los reintentos, la caja emite ERR:timeout, abandona el EVT y sigue operando: el modo local y la señalización no se pierden, y el próximo cambio de modo genera tramas nuevas. Si el ACK llega tarde (durante los reintentos), simplemente limpia el pendiente y no se reenvía más. Este comportamiento está verificado por tests y se observa en la sesión simulada (Captura 4).

3. ¿Cómo aseguramos que los payloads numéricos (adc=NNNN, spd=NN) siempre tengan el ancho correcto aunque el valor sea chico (ej. adc=0023)?

Con formato de ancho fijo y relleno de ceros de `sprintf`: "param=%03u" para el parámetro (param=007) y "adc=%04u" para el ADC (adc=0023). El especificador `%0Nu` garantiza N dígitos independientemente del valor, y como los valores están acotados por construcción (param ≤ 255, adc ≤ 4095) nunca se excede el ancho. El test 'DAT:param=007 (ancho fijo)' cubre el caso límite.

4. ¿Qué decisión tomaste sobre el uso de timers? ¿Cuántos timers usaste y para qué?

Tres timers de hardware más el SysTick: TIM2 genera el PWM de 1 kHz del LED (duty por CCR2); TIM4 genera el tono de 2 kHz del buzzer (on/off por CCR1, frecuencia intacta); TIM3 es la base temporal de 1 ms que pide la consigna (muestreo de ADC cada 500 ms y anti-rebote de 50 ms). El SysTick queda reservado para el tick de FreeRTOS, del que derivan los tiempos 'blandos' (patrones de señalización cada 10 ms, timeout de ACK). Separar LED y buzzer en timers distintos evita la limitación de frecuencia compartida discutida en la Etapa 2, y mantener TIM3 dedicado hace la temporización de seguridad independiente del scheduler.

7.2 Evidencia mínima

El diagrama de estados es la Figura 4. Sesión de operación continua (~23 s) con todas las tramas relevantes, generada con la lógica real del firmware y el bridge simulado (evidencia/etapa3_sesion_simulada.txt):

```
[ 2.000] BOTON presionado (anti-rebote OK), pote=1523
[ 2.000] PILL->BRIDGE @0E:EVT:cmd_deploy:0C
[ 2.000] PILL->BRIDGE @0F:STS:mode=DEPLOY:17
[ 2.000] PILL->BRIDGE @0D:DAT:param=094:4A
[ 2.040] BRIDGE->PILL @06:ACK:ok:4B
[ 6.040] BOTON presionado (anti-rebote OK), pote=3800
[ 6.040] PILL->BRIDGE @0E:EVT:cmd_return:0D
[ 6.040] PILL->BRIDGE @0D:STS:mode=STOP:06
```

```
[ 6.040] PILL->BRIDGE @0D:DAT:param=236:40
[ 6.075] BRIDGE->PILL @06:ACK:ok:4B
[10.075] BOTON presionado / bridge desconectado (falla)
[10.075] PILL->BRIDGE @0E:EVT:cmd_patrol:13
[10.075] PILL->BRIDGE @0F:STS:mode=PATROL:08
[10.075] PILL->BRIDGE @0D:DAT:param=031:45
[10.575] PILL->BRIDGE @0E:EVT:cmd_patrol:13 (reenvio 1)
[11.075] PILL->BRIDGE @0E:EVT:cmd_patrol:13 (reenvio 2)
[11.295] BRIDGE->PILL @06:ACK:ok:4B (bridge vuelve)
[14.295] BOTON: cmd_stop + mode=STOP + param=127 (ACK ok)
[18.333] BOTON presionado / bridge desconectado hasta agotar reintentos
[18.333] PILL->BRIDGE @0E:EVT:cmd_deploy:0C (+ STS + DAT)
[18.833..20.833] 5 reenvios de @0E:EVT:cmd_deploy:0C cada 500 ms
[21.333] PILL->BRIDGE @0B:ERR:timeout:4C
```

Captura 4 — Sesión continua: ciclo de modos, ACK, reintentos y ERR:timeout.

Pendiente para la demo presencial: foto/video del montaje físico completo (Blue Pill + protoboard + periféricos) y captura del monitor serie real, a registrar con el hardware armado.

8. Etapa 4 — Integración con el robot

Esta etapa se completa en el laboratorio con el bridge y el robot de la cátedra. El plan de integración y el escenario de falla ya ensayado quedan documentados a continuación; las respuestas que dependen de mediciones en vivo se dejan planteadas con su método de medición.

8.1 Preguntas obligatorias

1. ¿Qué tópicos de ROS 2 publica el bridge a partir de las tramas que envía tu Blue Pill?

Los define la especificación del bridge de la cátedra (a confirmar en el laboratorio con `ros2 topic list` y `ros2 topic echo`). Lo esperable según la arquitectura del TP: los `EVT:cmd_*` se traducen a comandos de comportamiento del robot (patrullar, detenerse, posición de guardia, volver a base), los `STS:mode=*` a un tópico de estado de la caja, y los `DAT:param=NNN` a un tópico de datos auxiliares. Nuestro firmware es agnóstico: sólo garantiza tramas sintácticamente válidas, que es el contrato con el bridge.

2. ¿Qué latencia observás entre que ocurre un evento físico y la reacción del robot?

A medir en el laboratorio filmando botón y robot en el mismo cuadro. El presupuesto de latencia del lado Pill es conocido: 50 ms de anti-rebote (dominante) + <10 ms de scheduling de `task_app` + ~2 ms de transmisión de la trama de 22 bytes a 115200 baudios ≈ 60 ms. Lo que se suma por encima corresponde al bridge, a ROS 2 y a la mecánica del robot; esperamos un total percibido en el orden de 100–500 ms.

3. En el escenario de falla que probaste, ¿la Blue Pill se recupera automáticamente o requiere intervención manual? ¿Es ese el comportamiento deseado?

Se recupera sola. En el escenario 'bridge desconectado' (Captura 4) hay dos desenlaces, ambos automáticos: si el bridge vuelve durante los reintentos, el ACK tardío limpia el pendiente y la operación continúa sin pérdida de estado; si no vuelve, tras 5 reintentos la caja emite `ERR:timeout`, desiste de ese EVT y sigue funcionando, de modo que el siguiente cambio de modo vuelve a intentar la comunicación. Creemos que es el comportamiento correcto para una caja de comandos: no debe bloquearse esperando a un bridge caído ni perder su estado local; la única mejora discutible sería una señalización local visible (p. ej. un patrón de LED de 'sin enlace') mientras haya timeouts consecutivos.

4. Si tuvieras que agregar un nuevo comando de robot sin modificar el protocolo, ¿cómo lo harías?

El protocolo transporta payloads ASCII opacos, así que alcanza con definir un payload nuevo dentro del tipo EVT existente, por ejemplo EVT:cmd_dock. En la Pill son tres cambios de tabla: el string en k_evt_payload[], el modo en el enum y su patrón/pitidos en signals; del lado del bridge, mapear el payload nuevo al servicio ROS 2 correspondiente. El framing, el checksum, el parser y el manejo de ACK no se tocan: esa extensibilidad es justamente el beneficio de haber separado transporte (protocol/parser) de semántica (command_box).

8.2 Escenario de falla probado

Escenario: pérdida del bridge en el momento exacto en que el operador cambia de modo (equivalente a desconectar el cable UART). Se ensayó con la lógica real del firmware en dos variantes (Captura 4): caída transitoria — el EVT:cmd_patrol se reenvía a los 500 ms y 1000 ms, el bridge vuelve y su ACK:ok tardío cierra el pendiente; y caída total — EVT:cmd_deploy se reenvía 5 veces y a los 3 s la caja emite ERR:timeout y desiste. En ambos casos el sistema queda operativo y el estado local es consistente; no hace falta reset ni intervención manual. Con hardware, el mismo escenario se reproduce desconectando el jumper de RX del USB-UART, y la herramienta tools/bridge_sim.py --no-ack permite forzarlo de manera controlada.

Pendiente para la sesión de laboratorio: video del robot reaccionando a los cambios de modo y captura de los tópicos ROS 2 en paralelo con el monitor serie.

9. Reflexión técnica del grupo

Lo más valioso del TP fue comprobar que la disciplina de capas paga: como command_box.c y signals.c son lógica pura sin registros ni RTOS, pudimos escribir 53 tests que corren en la PC y depurar el ciclo de modos, los reintentos de ACK y los patrones de señalización antes de tocar la placa. El firmware que llega al hardware ya tiene la parte difícil validada, y los errores que queden serán de configuración de periféricos, no de lógica.

Del protocolo aprendimos que la robustez no está en el caso feliz sino en los bordes: LL inconsistente, checksum malo, tramas pegadas, '@' en medio de una trama rota. La FSM byte a byte resuelve todos esos casos con memoria constante, algo que un readline() nunca daría. También entendimos en la práctica por qué las ISR deben ser mínimas (encolar y salir): la primera versión de la lectura de ADC dentro de la ISR de TIM3 nos obligó a razonar el peor caso (~6 µs de conversión) para justificar que era aceptable.

Sobre los timers, la lección fue concreta: los canales de un timer comparten frecuencia, así que 'un timer por función' (PWM de brillo, tono, base temporal) es más simple y más flexible que intentar multiplexar. Y sobre la seguridad, arrancar la caja en STOP y tratar el timeout de ACK como un evento reportable (ERR:timeout) en lugar de un bloqueo nos parece la decisión de diseño más importante del trabajo: el sistema degrada avisando, sin perder el control local.

Anexo A — Tramas de referencia de la Variante 3

Generadas con protocol_encode_frame() y verificadas por el arnés de tests (los cuatro ejemplos del enunciado también se reproducen exactamente):

```
@0E:EVT:cmd_patrol:13    @0F:STS:mode=PATROL:08
@0C:EVT:cmd_stop:19      @0D:STS:mode=STOP:06
@0E:EVT:cmd_deploy:0C    @0F:STS:mode=DEPLOY:17
```

```
@0E:EVT:cmd_return:0D    @0D:DAT:param=128:4C
@0D:DAT:param=007:40    @06:ACK:ok:4B
@0B:ERR:timeout:4C      @0A:ERR:bounds:35
```

Anexo B — Contenido de la entrega

- firmware/ — código fuente completo (FreeRTOS + libopencm3) con Makefile funcional; third_party/README.md explica cómo obtener las dependencias.
- tests/ — arnés de 53 tests de host (make run), recorrido de FSM (make fsm) y sesión simulada (make sim).
- tools/bridge_sim.py — bridge simulado por puerto serie: responde ACK:ok a cada EVT; con --no-ack fuerza el escenario de timeout.
- evidencia/ — capturas citadas en este informe (tests de protocolo, recorrido de FSM, lecturas de ADC, sesión simulada).
- diagramas/ — figuras en PNG (conexión, estados, FSM, tareas, patrones).

Las capturas de este informe fueron generadas ejecutando la lógica real del firmware en la PC (tests de host y bridge simulado). Las evidencias que requieren el montaje físico y el robot (fotos, video de demo, tópicos ROS 2, latencia) están marcadas como pendientes y se registran en la sesión de laboratorio.